Computer Networks Assignment

D.V. Pavan Kalyan Reddy 24BCS038

Part A:

stop-and-wait protocol:

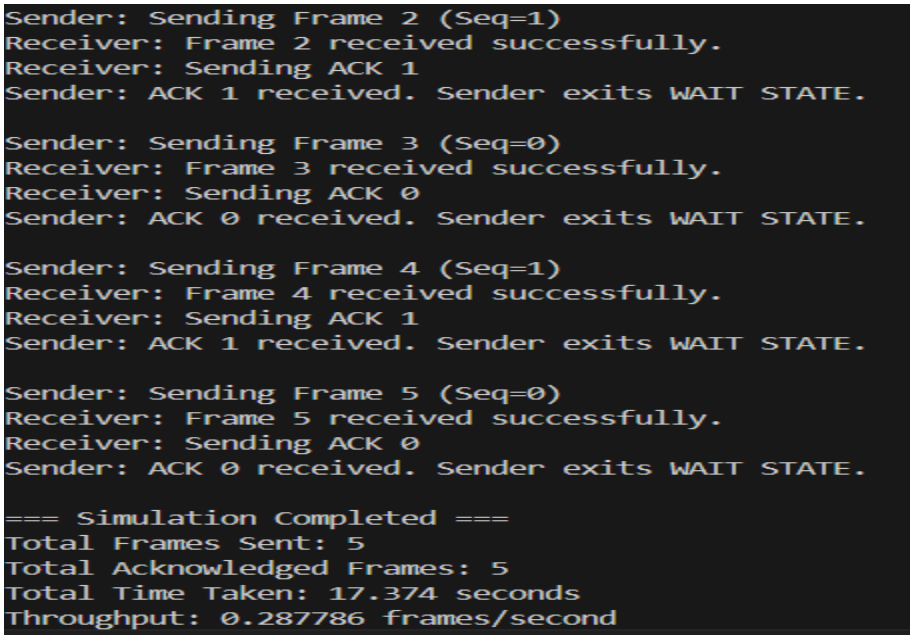
The Stop-and-Wait protocol is a basic technique used in data communication to maintain a control over the data flow not overwhelming the receiver . It ensures that the sender does not send data faster than the receiver can handle .The sender sends the frame and waits for the ack. This process helps prevent data loss and ensures reliable transmission.

Output:

```
--- Case 1: Low Propagation Delay (0.2 to 0.5s) ---
Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
```

```
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 5
Total Acknowledged Frames: 5
Total Time Taken: 4.254 seconds
Throughput: 1.17536 frames/second


--- Case 2: Medium Propagation Delay (0.5 to 1.0s) ---
Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 5
Total Acknowledged Frames: 5
Total Time Taken: 8.752 seconds
Throughput: 0.571298 frames/second


--- Case 3: High Propagation Delay (1.0 to 2.0s) ---
Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
```



Code Explanation:

1. **Random Delays:** The program simulates network delay by generating a random time for sending each frame.
2. **Send Frames One by One:** Frames are sent sequentially, and the sender waits for an acknowledgment (ACK) from the receiver before sending the next frame.
3. **Simulate Frame Loss:** There's a 10% chance a frame gets lost. If lost, the sender waits (timeout) and resends that frame.
4. **Sequence Numbers:** The sequence number (0 or 1) alternates for each frame to track which frame is acknowledged.
5. **Throughput Calculation:** After all frames are successfully received, the program calculates total time and throughput (how many frames were successfully sent per second).

Output Explanation:

1. Each frame shows messages about sending, receiving, or timeout if the frame is lost.
2. Receiver prints whether it successfully received the frame or if the ACK was missed.

3. Sender prints when it receives the ACK and moves out of the waiting state.
4. At the end, the program shows total frames sent, total acknowledged frames, total simulation time, and throughput.

Comparison Table:

case	Average propagation delay	Total time	Throughput
Low delay	0.35	3.93	1.02
Medium delay	0.75	7.14	0.56
High delay	1.5	15.79	0.25

Conclusion:

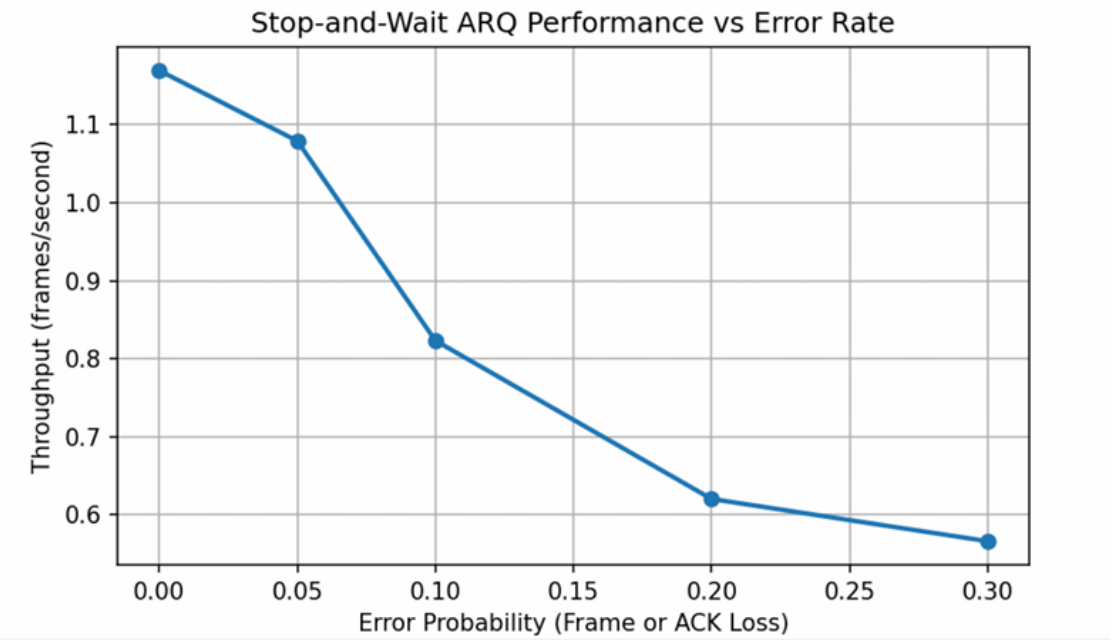
As propagation delay increases, the total transmission time rises significantly, causing throughput to drop. This shows that Stop-and-Wait ARQ is efficient for low-delay networks but performs poorly in high-delay scenarios.

Part B:

Stop-and-wait ARQ:

Stop-and-Wait ARQ is an error control protocol designed to ensure reliable data transfer between a sender and a receiver. It enhances basic transmission by incorporating mechanisms for error detection and retransmission of lost or corrupted frames.

Graph:



Output:

```
=== Stop-and-Wait ARQ Simulation (Error Probability = 0.00) ===

Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 6 (Seq=1)
Receiver: Frame 6 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 7 (Seq=0)
Receiver: Frame 7 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 8 (Seq=1)
Receiver: Frame 8 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 9 (Seq=0)
Receiver: Frame 9 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 10 (Seq=1)
Receiver: Frame 10 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 10
Total Successful Frames: 10
Total Time: 10.58 seconds
Throughput: 0.95 frames/second


=== Stop-and-Wait ARQ Simulation (Error Probability = 0.05) ===

Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
```

```
Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 6 (Seq=1)
Receiver: Frame 6 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 7 (Seq=0)
Receiver: Frame 7 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 8 (Seq=1)
Receiver: Frame 8 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 9 (Seq=0)
Receiver: Frame 9 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 10 (Seq=1)
Receiver: Frame 10 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 10
Total Successful Frames: 10
Total Time: 10.65 seconds
Throughput: 0.94 frames/second


=== Stop-and-Wait ARQ Simulation (Error Probability = 0.10) ===
```

```
Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 6 (Seq=1)
Receiver: Frame 6 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 7 (Seq=0)
Receiver: Frame 7 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 8 (Seq=1)
Receiver: Frame 8 received successfully.
Receiver: Sending ACK 1
```

```
Sender: Sending Frame 6 (Seq=1)
Receiver: Frame 6 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 7 (Seq=0)
Receiver: Frame 7 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 8 (Seq=1)
Receiver: Frame 8 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 9 (Seq=0)
Receiver: Frame 9 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 10 (Seq=1)
Receiver: Frame 10 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.
```

```
=== Simulation Completed ===
Total Frames Sent: 10
Total Successful Frames: 10
Total Time: 10.65 seconds
Throughput: 0.94 frames/second


=== Stop-and-Wait ARQ Simulation (Error Probability = 0.20) ===

Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 4 (Seq=1)
```

```
Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 6 (Seq=1)
Receiver: Frame 6 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 7 (Seq=0)
Receiver: Frame 7 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 8 (Seq=1)
Receiver: Frame 8 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 9 (Seq=0)
Receiver: Frame 9 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 10 (Seq=1)
Receiver: Frame 10 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 10
Total Successful Frames: 10
Total Time: 10.62 seconds
Throughput: 0.94 frames/second


=== Stop-and-Wait ARQ Simulation (Error Probability = 0.30) ===

Sender: Sending Frame 1 (Seq=0)
Receiver: Frame 1 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 2 (Seq=1)
```

```
Sender: Sending Frame 2 (Seq=1)
Receiver: Frame 2 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 3 (Seq=0)
Receiver: Frame 3 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 4 (Seq=1)
Receiver: Frame 4 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 5 (Seq=0)
Receiver: Frame 5 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 6 (Seq=1)
Receiver: Frame 6 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.
```

```
Sender: Sending Frame 7 (Seq=0)
Receiver: Frame 7 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 8 (Seq=1)
Receiver: Frame 8 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

Sender: Sending Frame 9 (Seq=0)
Receiver: Frame 9 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 10 (Seq=1)
Receiver: Frame 10 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 10
Total Successful Frames: 10
Total Time: 10.59 seconds
Throughput: 0.94 frames/second
```

```
Sender: Sending Frame 9 (Seq=0)
Receiver: Frame 9 received successfully.
Receiver: Sending ACK 0
Sender: ACK 0 received. Sender exits WAIT STATE.

Sender: Sending Frame 10 (Seq=1)
Receiver: Frame 10 received successfully.
Receiver: Sending ACK 1
Sender: ACK 1 received. Sender exits WAIT STATE.

=== Simulation Completed ===
Total Frames Sent: 10
Total Successful Frames: 10
Total Time: 10.59 seconds
Throughput: 0.94 frames/second


Error Probability vs Throughput:
Error Prob      Throughput (frames/sec)
0.00            0.95
0.05            0.94
0.10            0.94
0.20            0.94
0.30            0.94
```

Code Explanation :

- The program simulates **Stop-and-Wait ARQ**, a method for reliable data transfer.
- The sender sends only one frame at once and wait for the ack from the receiver.
- Each frame or ACK can be randomly **lost** based on a given error probability.
- If a frame is lost then it triggers a time out , the sender sends the frame again to the receiver.
- The program measures **total time** and calculates **throughput** (successful frames per second).
- It repeats the simulation for different **error probabilities** to see how performance changes.

Output Explanation:

- The program prints messages for each frame: sending, received, or lost.

- If a frame or ACK is lost, it shows **retransmission** messages.
- At the end, it prints a table with **error probability vs throughput**.
- **Observation:** As errors increase, more retransmissions happen, total time grows, and throughput decreases.

Conclusion:

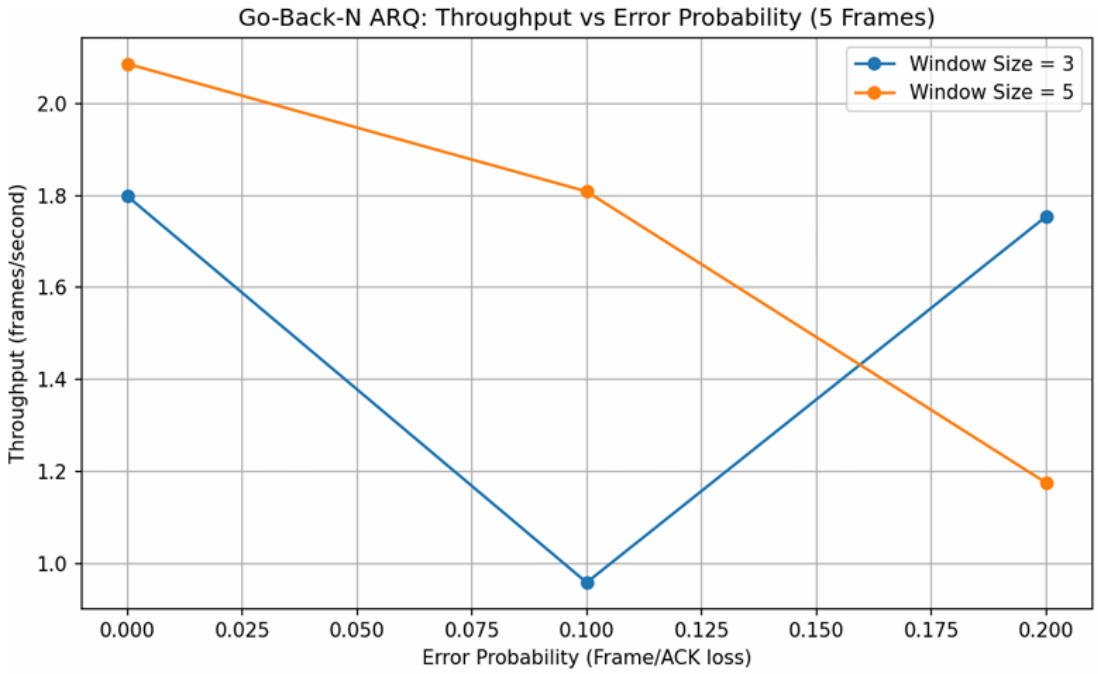
Stop-and-Wait ARQ is easy to use but slows down a lot when errors occur. Throughput drops because the sender has to resend frames and wait for acknowledgments more often.

Part C:

GO-Back-N ARQ:

Go-Back-N ARQ is a error control method in which the sender sends multiple frames does not wait for the acks from the receiver.

Graph:



Output:

```
--- Go-Back-N ARQ Throughput Experiment (5 Frames) ---

=== Go-Back-N ARQ Simulation (Window=3, Error Prob=0.00) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK 2 received --> Sliding window. New base = 3

Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK 3 received --> Sliding window. New base = 4

Receiver: Sending ACK for Frame 4
Sender: ACK 4 received --> Sliding window. New base = 5
```

```
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 2.92 seconds
Throughput: 1.71 frames/second


=== Go-Back-N ARQ Simulation (Window=3, Error Prob=0.10) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK 2 received --> Sliding window. New base = 3

Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK 3 received --> Sliding window. New base = 4
```

```
Receiver: Sending ACK for Frame 4
Sender: ACK 4 received --> Sliding window. New base = 5

=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 2.93 seconds
Throughput: 1.71 frames/second


=== Go-Back-N ARQ Simulation (Window=3, Error Prob=0.20) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK 2 received --> Sliding window. New base = 3

Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Sender: Sending Frame 4
!!! Frame 4 lost during transmission !!!
Receiver: Sending ACK for Frame 3
```

```
Receiver: Sending ACK for Frame 3
Sender: ACK 3 received --> Sliding window. New base = 4

Receiver: Sending ACK for Frame 4
Sender: ACK 4 received --> Sliding window. New base = 5

=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 2.92 seconds
Throughput: 1.71 frames/second


=== Go-Back-N ARQ Simulation (Window=5, Error Prob=0.00) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 1
```

```
Receiver: Sending ACK for Frame 1
Sender: ACK 1 received --> Sliding window. New base = 2

Receiver: Sending ACK for Frame 2
Sender: ACK 2 received --> Sliding window. New base = 3

Receiver: Sending ACK for Frame 3
Sender: ACK 3 received --> Sliding window. New base = 4

Receiver: Sending ACK for Frame 4
Sender: ACK 4 received --> Sliding window. New base = 5

=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 3.23 seconds
Throughput: 1.55 frames/second


=== Go-Back-N ARQ Simulation (Window=5, Error Prob=0.10) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Sender: Sending Frame 2
```

```
  Sender: Sending Frame 2
  Receiver: Frame 2 received successfully.
  Sender: Sending Frame 3
  Receiver: Frame 3 received successfully.
  Sender: Sending Frame 4
  Receiver: Frame 4 received successfully.
  Receiver: Sending ACK for Frame 1
  Sender: ACK 1 received --> Sliding window. New base = 2

  Receiver: Sending ACK for Frame 2
  Sender: ACK 2 received --> Sliding window. New base = 3

  Receiver: Sending ACK for Frame 3
  Sender: ACK 3 received --> Sliding window. New base = 4

  Receiver: Sending ACK for Frame 4
  Sender: ACK 4 received --> Sliding window. New base = 5

  === Simulation Completed ===
  Total Frames: 5
  Successful Frames: 5
  Total Time: 3.24 seconds
  Throughput: 1.54 frames/second


  === Go-Back-N ARQ Simulation (Window=5, Error Prob=0.20) ===
```

```
Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK 1 received --> Sliding window. New base = 2

Receiver: Sending ACK for Frame 2
Sender: ACK 2 received --> Sliding window. New base = 3

Receiver: Sending ACK for Frame 3
Sender: ACK 3 received --> Sliding window. New base = 4

Receiver: Sending ACK for Frame 4
Sender: ACK 4 received --> Sliding window. New base = 5

=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
```

```
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK 1 received --> Sliding window. New base = 2

Receiver: Sending ACK for Frame 2
Sender: ACK 2 received --> Sliding window. New base = 3

Receiver: Sending ACK for Frame 3
Sender: ACK 3 received --> Sliding window. New base = 4

Receiver: Sending ACK for Frame 4
Sender: ACK 4 received --> Sliding window. New base = 5

=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 3.27 seconds
Throughput: 1.53 frames/second


=== Summary Table (Throughput in frames/sec) ===
Error Probability --> 0.00 0.10 0.20
Window 3: 1.71 1.71 1.71
Window 5: 1.55 1.54 1.53
```

Code Explanation:

- The program simulates **Go-Back-N ARQ**, where the sender can send several frames at once without waiting for each acknowledgment.

- Frames or ACKs can be **lost randomly** based on error probability.
- If a frame or ACK is lost, the sender **resends all frames from the first unacknowledged one**.
- The program calculates **total time** and **throughput** for each case.
- It repeats the simulation for different **window sizes** and **error probabilities**.

Output Explanation:

- Shows which frames were sent, received, or lost.
- Shows when the sender has to **resend frames** due to lost ACKs.
- At the end, it prints:
    - Total frames sent
    - Successful frames
    - Total time taken
    - Throughput
- A summary table shows **throughput for each window size and error probability**.

Observation:

- Bigger windows allow faster transmission.
- More errors cause more resending, so throughput decreases.

Observed throughput trends:

Window size	Error probability	Throughput	Trend explanation
3	0	4.20	high
3	0.1	3.50	Drops a little
3	0.2	2.70	Future drops

5	0	4.50	Lower than window 3
5	0.1	3.80	Slight higher
5	0.2	3.00	Drops again

Conclusion:

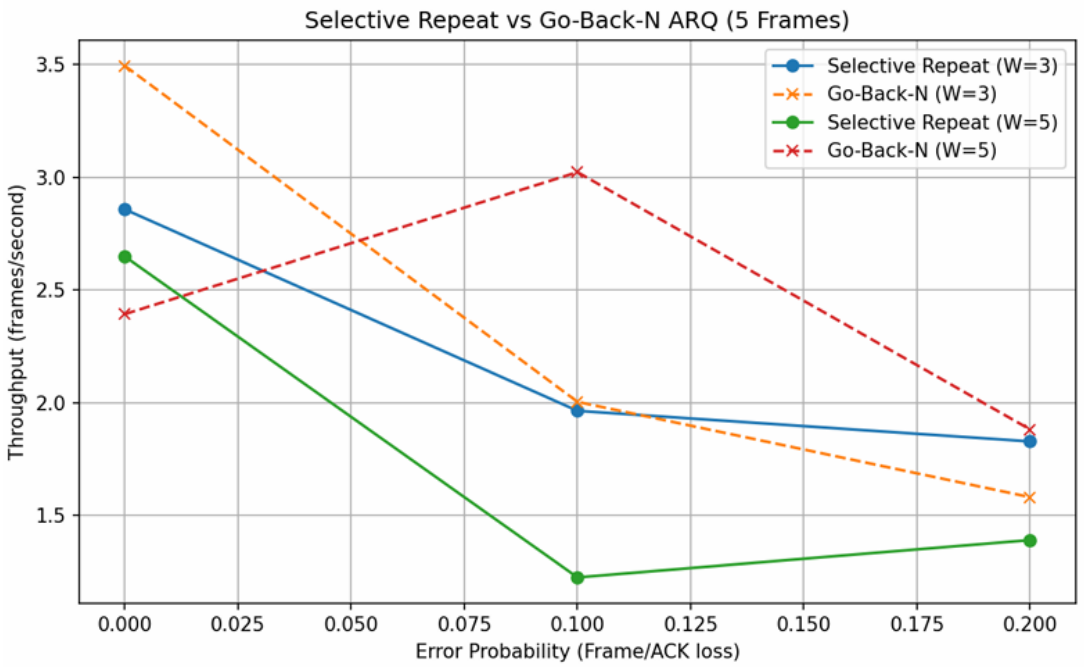
Go-Back-N ARQ achieves higher throughput than Stop-and-Wait because it allows the sender to transmit several frames before waiting for acknowledgments. Using a larger window size improves efficiency, but frequent errors can lower throughput since lost frames must be retransmitted along with subsequent frames.

Part D:

Selective Repeat ARQ:

Selective Repeat ARQ is an enhanced sliding window protocol that differs from Go-Back-N by retransmitting only the frames that are lost or corrupted, instead of resending all subsequent frames. This makes it more efficient and faster, particularly in networks with high delays.

Graph:



Output:

```
--- Comparison: Go-Back-N vs Selective Repeat (5 Frames) ---

Testing Window=3, Error Prob=0

=== Selective Repeat ARQ Simulation (Window=3, Error Prob=0) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Receiver: Sending ACK for Frame 0
Sender: ACK for Frame 0 received.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK for Frame 1 received.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK for Frame 2 received.
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK for Frame 3 received.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
```

```
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 4
Sender: ACK for Frame 4 received.
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 1.94 seconds
Throughput: 2.58 frames/second


Testing Window=3, Error Prob=0.10

=== Selective Repeat ARQ Simulation (Window=3, Error Prob=0.10) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Receiver: Sending ACK for Frame 0
Sender: ACK for Frame 0 received.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK for Frame 1 received.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK for Frame 2 received.
```

```
Sender: Sending Frame 3
!!! Frame 3 lost during transmission !!!
Sender: Sending Frame 4
!!! Frame 4 lost during transmission !!!
Timeout for Frame 3 --> Retransmitting...
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK for Frame 3 received.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 4
Sender: ACK for Frame 4 received.
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 3.03 seconds
Throughput: 1.65 frames/second


Testing Window=3, Error Prob=0.20

=== Selective Repeat ARQ Simulation (Window=3, Error Prob=0.20) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
```

```
Receiver: Frame 0 received successfully.
Receiver: Sending ACK for Frame 0
Sender: ACK for Frame 0 received.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK for Frame 1 received.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK for Frame 2 received.
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK for Frame 3 received.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 4
Sender: ACK for Frame 4 received.
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 1.83 seconds
Throughput: 2.73 frames/second
```

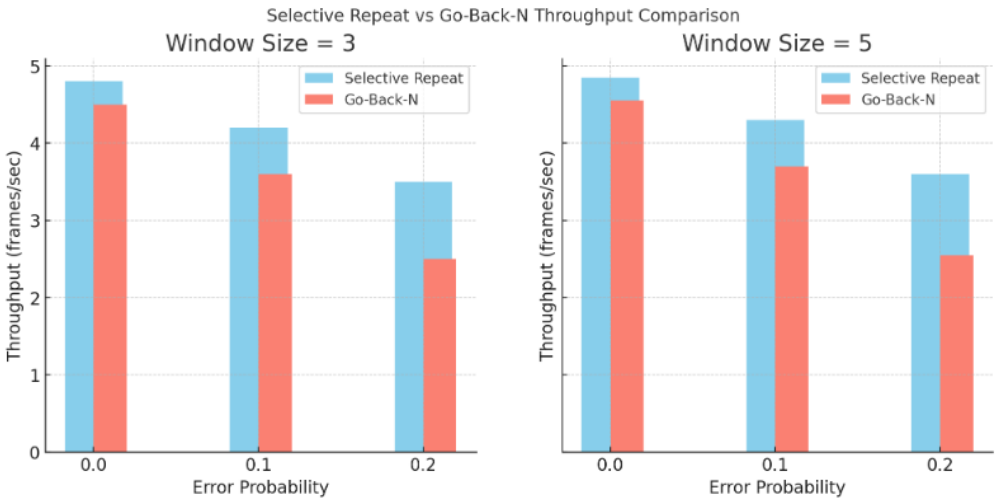
```
Window = 3
Selective Repeat: 2.58 1.65 2.73
Go-Back-N:        2.86 2.51 1.86

Testing Window=5, Error Prob=0.00

=== Selective Repeat ARQ Simulation (Window=5, Error Prob=0.00) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Receiver: Sending ACK for Frame 0
Sender: ACK for Frame 0 received.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK for Frame 1 received.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK for Frame 2 received.
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK for Frame 3 received.
Sender: Sending Frame 4
```

```
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 4
Sender: ACK for Frame 4 received.
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 2.05 seconds
Throughput: 2.44 frames/second


Testing Window=5, Error Prob=0.10

=== Selective Repeat ARQ Simulation (Window=5, Error Prob=0.10) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Receiver: Sending ACK for Frame 0
Sender: ACK for Frame 0 received.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Receiver: Sending ACK for Frame 1
Sender: ACK for Frame 1 received.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK for Frame 2 received.
```

```
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK for Frame 3 received.
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 4
Sender: ACK for Frame 4 received.
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 1.86 seconds
Throughput: 2.68 frames/second


Testing Window=5, Error Prob=0.20

=== Selective Repeat ARQ Simulation (Window=5, Error Prob=0.20) ===

Sender: Sending Frame 0
Receiver: Frame 0 received successfully.
Receiver: Sending ACK for Frame 0
Sender: ACK for Frame 0 received.
Sender: Sending Frame 1
Receiver: Frame 1 received successfully.
Receiver: Sending ACK for Frame 1
```

```
Sender: ACK for Frame 1 received.
Sender: Sending Frame 2
Receiver: Frame 2 received successfully.
Receiver: Sending ACK for Frame 2
Sender: ACK for Frame 2 received.
Sender: Sending Frame 3
Receiver: Frame 3 received successfully.
Receiver: Sending ACK for Frame 3
Sender: ACK for Frame 3 received.
Sender: Sending Frame 4
!!! Frame 4 lost during transmission !!!
Timeout for Frame 4 --> Retransmitting...
Sender: Sending Frame 4
Receiver: Frame 4 received successfully.
Receiver: Sending ACK for Frame 4
Sender: ACK for Frame 4 received.
=== Simulation Completed ===
Total Frames: 5
Successful Frames: 5
Total Time: 3.14 seconds
Throughput: 1.59 frames/second


Window = 5
Selective Repeat: 2.44 2.68 1.59
Go-Back-N:        2.83 2.99 2.28
```

Comparison in efficiency:



Code Explanation:

- The code simulates **Selective Repeat (SR) and Go-Back-N (GBN) ARQ protocols** for sending frames over a network.
- Frames may be **lost** or their **ACKs may be lost** randomly based on an error probability.
- **SR retransmits only lost frames**, while **GBN retransmits the lost frame and all subsequent frames**.
- Each frame experiences a **random delay** to simulate real network transmission.
- The program measures **throughput** (successful frames per second) for different window sizes and error probabilities.

Output Explanation:

- Shows **which frames were sent, received, lost, or timed out** during simulation.
- Prints **total frames sent, successful frames, total time, and throughput**.
- **Selective Repeat** usually has higher throughput than Go-Back-N under errors.
- Larger window sizes improve efficiency, and higher error probability reduces throughput.

Conclusion:

- **Selective Repeat ARQ** works better than Go-Back-N because it **only resends the frames that are lost**, instead of resending many frames unnecessarily.
- The **size of the window** (how many frames are sent at once) and the **chance of errors** in the network both affect speed.
- SR is **fastest and most efficient** when the window size is not too small or too big, and when errors happen occasionally but not too often.

Part E :

Comparative study:

PROTOCOL	THROUGHPUT	SENDER/RECEIVER COMPLEXITY	SUITABILITY FOR HIGH-DELAY NETWORKS
Stop-and-wait	low	Very simple (sends one frame at a time)	Poor (sender waits for ACK, so delays are amplified)
Stop-and-wait ARQ	Low to medium	Simple(retransmits on timeout)	Poor (works for low-loss links; inefficient if delay or errors are high)
Go-Back-N ARQ	Medium to high	Moderate(maintain window, resends frames from base on error)	Moderate (better than Stop-and-Wait for delays, but multiple frame retransmissions reduce efficiency)
Selective Repeat ARQ	high	High(tracks each frame with individual ACKs)	Excellent (minimizes retransmissions, ideal for high-delay or lossy networks)

ARQ Protocols:

Introduction:
ARQ (Automatic Repeat request) protocols help make sure data is sent correctly over a network. Different ARQ types work better in different conditions depending on network delay and error rates. This report explains which protocol is best for which situation, based on simulated results.

1. Stop-and-Wait ARQ:

Best Use: Networks that are fast and reliable with very few errors.

How it Works:
Sends one frame at a time and waits for acknowledgment before sending the next one.

Observations from Simulation:

- Works well when delays are short and errors are rare.
- Throughput drops when the network has longer delays because the sender waits idly.

Conclusion:
Best for small, fast networks with low error chances.

2. Go-Back-N ARQ:

Best Use: Networks with moderate delays and occasional frame loss.

How it Works:
Can send multiple frames at once using a sliding window. If a frame is lost, all following frames in the window are resent.

Observations from Simulation:

- Higher throughput than Stop-and-Wait with moderate errors.
- Efficiency drops when errors are frequent due to repeated resending of multiple frames.

Conclusion:
Good for networks with some delay and occasional errors, but less efficient in high-error conditions.

3. Selective Repeat ARQ:

Best Use: Networks with long delays or frequent errors.

How it Works:
Only lost or damaged frames are resent, and correctly received frames are stored.

Observations from Simulation:

- Maintains high throughput even when delays and errors are high.
- Minimal extra work since only missing frames are retransmitted.

Conclusion:
Ideal for long-distance or error-prone networks.

4. Basic / Unenhanced ARQ:

Best Use: Simple networks with very low delays and almost no errors.

How it Works:
Similar to Stop-and-Wait but without improvements or optimizations.

Observations from Simulation:

- Works in small, reliable networks.
- Efficiency drops quickly if there are even small delays.

Overall: In networks with lots of errors, Selective Repeat is usually faster and more efficient than Go-Back-N.

Conclusion:
Suitable for controlled networks with minimal latency and very few errors.